

DYNAMIC APPLICATION SECURITY TESTING

ReNew Solar Marketing Website

Runtime Security Assessment Report

CONFIDENTIAL. This document contains a working exploit proof-of-concept and runtime evidence that could assist an attacker if disclosed. Distribute only to personnel responsible for remediating the systems described.

TARGET

ReNew Solar Marketing Site

TEST DATE

17 August 2026

PRODUCTION TARGET

https://re-new-mocha.vercel.app/

BUILD TESTED

a4f4020 (main)

TEST TYPE

Dynamic — running application

INSTRUMENTATION

Chrome DevTools Protocol

EXPLOIT CONFIRMED

1 — clickjacking (PoC included)

REPORT VERSION

1.1 — Post-remediation

REMEDATION APPLIED

17 August 2026

STATUS

4 verified · 3 pending deploy · 1 mitigated

1 Executive Summary

A dynamic application security test was performed against the ReNew Solar marketing website in its running state. Unlike the static analysis in the companion SAST report, this engagement executed the application in an instrumented browser and observed what it *actually does* — intercepting every network egress channel, injecting live attack payloads, driving the forms as a user would, and building a working exploit for the one issue that proved exploitable.

CONFIRMED EXPLOIT

Clickjacking is not theoretical on this target — it was exploited. The live production site was successfully embedded in an iframe from an unrelated origin (`file://`) with an attacker-controlled overlay rendered on top of it. A screenshot of the working proof-of-concept against production is reproduced in Section 5 as DAST-01. The site has no framing response header *and* no JavaScript frame-busting fallback — verified as zero occurrences across both source and compiled bundles.

The second material result is runtime confirmation of the enquiry-form defect. With `fetch`, `XMLHttpRequest` and `navigator.sendBeacon` all instrumented before interaction, a complete contact form was filled and submitted. The application displayed "*Thanks — we'll get back to you within 24 hours*" while producing **zero network egress on every channel**. This is direct observational proof, not inference from code.

Against that, the application's defensive behaviour under attack was **strong**. Twelve injection payloads — XSS, encoded XSS, path traversal, prototype pollution and a valid-prefix bypass attempt — were fired at the only attacker-controllable input in the application. All twelve were rejected with no script execution, no DOM injection and no prototype pollution. The platform additionally proved to have an active bot-mitigation layer, which triggered on the scripted probing during this test.

Findings by severity — as tested (v1.0)

0 CRITICAL	1 HIGH	3 MEDIUM	3 LOW	2 INFORMATIONAL
---------------	-----------	-------------	----------	--------------------

POST-REMEDIATION STATUS (V1.1)

3 findings re-tested and verified fixed in an instrumented browser · **3 fixed in configuration but pending deployment**, including the clickjacking exploit · **1 mitigated** · 2 needed no action. **The confirmed exploit should be assumed live until the PoC is re-run against the deployed site.** Detail in Section 1A.

RELATIONSHIP TO THE VAPT AND SAST REPORTS

This is the third report in a set of three covering the same target on the same date. The VAPT established the external attack surface; the SAST reviewed the source; this DAST report tests the running system. Several findings appear in more than one report because independent methods reached them independently — that convergence is itself a confidence signal. **Severities are consistent across all three documents, and each underlying defect should be remediated once.** Where DAST adds something the other two could not, it is stated explicitly in the finding.

1A Remediation Status & Retest

Remediation was carried out the same day version 1.0 was delivered, and the behavioural fixes were **re-tested dynamically** using the same instrumented-browser method as the original engagement. The findings in Section 4 are left as written — they describe the application as tested — and this section records the retest outcome.

ID	SEVERITY	FINDING	STATUS	RETEST OUTCOME
DAST-02	HIGH	Zero egress; success shown regardless	MITIGATED	Promise removed — "within 24 hours" no longer present. Egress still zero; endpoint deferred by client
DAST-01	MEDIUM	Clickjacking — confirmed exploit	FIXED — DEPLOY	X-Frame-Options: DENY + frame-ancestors 'none' authored; PoC must be re-run post-deploy
DAST-03	MEDIUM	Required-field validation passes when empty	VERIFIED FIXED	checkValidity() → false when empty, true once selected
DAST-04	MEDIUM	Identical output for every state	VERIFIED FIXED	Four states → four distinct tariffs and figures
DAST-05	LOW	Numeric bounds unenforced	VERIFIED FIXED	Clamped at 100,000 units with a visible cap notice
DAST-06	LOW	Security headers absent on all routes	FIXED — DEPLOY	Full header set in vercel.json ; applied at deploy time
DAST-07	LOW	Wildcard CORS	FIXED — DEPLOY	Scoped to a single origin
DAST-08	INFO	RSC payloads publicly retrievable	NO ACTION	Expected artifact; no sensitive data
DAST-09	INFO	Shared wildcard certificate	NO ACTION	Correct TLS config; revisit on domain move

1A.1 Retest evidence

DAST-04 — the state selector now drives the calculation. Compare against the identical-output table in Section 4:

		# was: identical for all four
₹11.00	₹66,000	
₹5.20	₹31,200	
₹7.10	₹42,600	
₹8.00	₹48,000	
distinctUnitCosts: 4	distinctAnnualFigures: 4	FIXED: true

Each figure now matches the "correct annual" column computed independently in SAST-01 — the 53.8% Ladakh overstatement and the 27.3% Maharashtra understatement are both resolved.

DAST-03 — the mandatory dropdown now participates in constraint validation:

```

                                # was "hidden" - barred from validation
                                false # was true - form submitted regardless
                                true
"FIXED": true }

```

DAST-05 — out-of-range input is clamped and the user is told:

```

₹28,80,00,000    yes # was ₹28,79,99,99,97,120
₹28,80,00,000    yes

```

Injection resistance — 11-payload regression run after the validator refactor:

```

"handlerIsLive": true,    # positive control still passes
"STILL_SECURE": true }

```

THREE FINDINGS REMAIN UNVERIFIED — DEPLOYMENT REQUIRED

DAST-01, DAST-06 and DAST-07 are fixed in configuration but not proven in production. `vercel.json` is applied by the hosting platform at deploy time and has no effect on a local instance, so none of these could be re-tested here. **DAST-01 in particular should be treated as still exploitable until the PoC is re-run against the deployed site and the iframe fails to render.** No deployment was performed as part of this remediation.

ACCEPTED RISK — CLIENT DECISION

DAST-02 is mitigated rather than closed. The runtime instrumentation still records `egressCount: 0` — form data continues to go nowhere. What changed is that the site no longer tells users it will respond within 24 hours; the confirmation now surfaces the phone number and email that actually reach the team. The endpoint was deferred by the client and remains the highest-priority open item across all three reports.

2 Methodology & Test Environment

2.1 Two test targets, and why

Dynamic testing was executed against two environments. Splitting them was necessary, and the split determines how each finding should be read.

ENVIRONMENT	TARGET	USED FOR
Production	<code>https://re-new-mocha.vercel.app/</code>	Response headers, TLS, HTTP methods, edge behaviour, reflection probing, and the clickjacking proof-of-concept — everything that depends on the real hosting layer.
Local instance	<code>127.0.0.1:8899</code>	Client-side runtime behaviour — payload injection, form instrumentation, calculator logic, storage inspection. Built from commit <code>a4f4020</code> via <code>next build</code> .

Client-side testing was moved local deliberately. The browser payload work required dozens of rapid interactions; running that volume against production would have been both discourteous and — as Section 2.3 describes — self-defeating. Because the deployment is a pure static export, the JavaScript bundle served locally is byte-identical in origin to production's, so client-side findings transfer directly. Anything header- or edge-dependent was tested against production only, and is labelled as such.

2.2 Instrumentation

Testing was driven through the Chrome DevTools Protocol against a real Chromium instance — a genuine browser engine executing the real React runtime, not an HTTP client simulating one. Egress interception was installed *before* any interaction:

```
// Installed prior to touching the form – captures every egress channel
window.__egress = [];
const origFetch = window.fetch;
window.fetch = (...a) => { window.__egress.push({ch:'fetch', url:String(a[0])}); return
origFetch(...a); };

const origOpen = XMLHttpRequest.prototype.open;
XMLHttpRequest.prototype.open = function(m, u, ...r) {
  window.__egress.push({ch:'xhr', method:m, url:String(u)}); return origOpen.call(this, m, u,
...r);
};

const origBeacon = navigator.sendBeacon?.bind(navigator);
if (origBeacon) navigator.sendBeacon = (u, d) => { window.__egress.push({ch:'beacon',
url:String(u)}); return origBeacon(u, d); };
```

TEST VALIDITY — CONTROL CASES

A negative result in dynamic testing is only meaningful if the code path under test actually executed. Every injection test in this report therefore carries a **positive control**: before firing payloads at the URL-fragment handler, two known-valid identifiers were submitted and confirmed to select two *different* tabs, proving the handler was live and processing input. Without that control, "no XSS fired" would be indistinguishable from "the feature never ran". Results are reported only where the control passed.

2.3 An observed platform control, and a testing constraint

Partway through HTTP method enumeration against production, the platform began returning `HTTP 403` with the header `x-vercel-mitigated: challenge` to the testing client. The block persisted for the source IP across a cooldown of more than seven minutes and affected all scripted requests, including ordinary `GET /`.

```
HTTP/2 403
x-vercel-mitigated: challenge
# ...still blocked after a 7-minute cooldown
403
```

Critically, a **real browser was unaffected throughout**. Loading the same URL in the instrumented Chromium returned the site normally, with the correct title and a full 220 KB document and no challenge interstitial:

```
"looksLikeChallenge": false
```

This has two consequences. It is recorded as a **positive security control** (Section 6) — the platform discriminates automated clients from real users and degrades only the former. And it is a stated **limitation**: remaining production checks were completed through the browser rather than the command line. Where a production result in this report was gathered before the mitigation engaged, or through the browser afterwards, the method is noted with the finding.

2.4 Rules of engagement

- Non-destructive throughout. No data modified, no denial-of-service attempted, no brute-force or credential attack (there is no authentication to attack).
- Request volume kept low; on encountering the mitigation, active scanning was **stopped immediately** and client-side testing relocated to the local instance.
- The clickjacking proof-of-concept was executed in a local browser only. It was never hosted, published, or made reachable by any third party.
- Injection payloads used inert markers (`window.__XSS = 1`) rather than `alert()` , to avoid modal dialogs blocking the automation.
- Third-party destinations linked from the site were not tested — out of scope.

2.5 Standards

OWASP Web Security Testing Guide v4.2 for test selection, OWASP Top 10:2021 for categorisation, CWE for weakness classification, CVSS v3.1 for scoring. Severities are environmentally adjusted for a static-export deployment with no server-side runtime, matching the framework used in the companion reports.

3 Findings Summary

ID	SEVERITY	FINDING	EVIDENCE TYPE	ENV
DAST-02	HIGH	Form submits with zero network egress; success shown regardless	Runtime observed	Local
DAST-01	MEDIUM	Clickjacking — working exploit against production	Exploit PoC	Production
DAST-03	MEDIUM	Required-field validation passes with the field empty	Runtime observed	Local
DAST-04	MEDIUM	Calculator returns identical output for every state	Runtime observed	Local
DAST-05	LOW	Declared numeric bounds not enforced at runtime	Runtime observed	Local
DAST-06	LOW	Security headers absent on all six routes tested	Response headers	Production
DAST-07	LOW	Wildcard CORS on every response including documents	Response headers	Production
DAST-08	INFO	React Server Component payloads publicly retrievable	Enumeration	Production
DAST-09	INFO	Shared platform wildcard certificate on preview domain	TLS inspection	Production

3.1 Attack simulations — all repelled

Active attacks executed against the running application. Every one failed, which is the result you want.

ATTACK	ATTEMPTS	RESULT	OBSERVATION
Reflected / DOM XSS via URL fragment	7	REPELLED	No execution, no DOM injection
Prototype pollution via fragment	2	REPELLED	<code>Object.prototype</code> intact
Path traversal via fragment	1	REPELLED	Rejected by allow-list
Valid-prefix allow-list bypass	1	REPELLED	Exact match enforced
Reflected XSS via URL path	2	REPELLED	404 body identical for all probes
Encoded path traversal at edge	1	REPELLED	Rejected with 400 before content
Dangerous HTTP methods	10	REPELLED	All mutating verbs 405
Cross-Site Tracing (TRACE)	1	REPELLED	405 — XST not possible
Clickjacking / UI redress	1	SUCCEEDED	See DAST-01

4 Detailed Findings

DAST-02 Form submits with zero network egress; success shown regardless

HIGH

CWE	OWASP	METHOD	CROSS-REF
CWE-670 — Always-Incorrect Control Flow	A04:2021 — Insecure Design	Runtime instrumentation	VAPT-02 / SAST-03

WHAT DYNAMIC TESTING ADDS

The SAST report inferred this defect from the submit handler. This test **observed it**: a complete, valid enquiry was submitted through the real UI with all three egress channels instrumented, and the data provably went nowhere.

EVIDENCE — OBSERVED RUNTIME RESULT

```
# Filled: name, company, phone, state – then submitted through the real UI

      "Thanks – we'll get back to you within 24 hours"
    []
    "egressCount": 0,
```

Independently corroborated by the DevTools network panel, which recorded no request of any kind following the submit event — the only entries present were React Server Component prefetches issued during initial page load.

The live DOM also shows neither form carries a destination at all:

```
2      # neither form has an action attribute
```

IMPACT

Every enquiry submitted through this site since launch has been silently discarded, while the user was shown an explicit 24-hour response commitment. For a marketing site whose primary purpose is lead capture, this defeats the asset. Because the success state is set unconditionally rather than derived from a response, the same control flow will continue to report success on server error even after an endpoint is added.

REMEDIATION

Restructure so the success state reflects a server acknowledgement, and build the receiving endpoint against the control set in Section 6.1 of the VAPT report. Full code is given in SAST-03. As an immediate measure, remove the 24-hour commitment from the UI copy and direct users to the published contact details, so enquiries stop being lost while the fix is built.

DAST-01 Clickjacking — working exploit against production

MEDIUM

CVSS V3.1	VECTOR	CWE	STATUS
4.3 — Medium	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	CWE-1021 — Improper UI Layer Restriction	Exploited — PoC below

WHAT DYNAMIC TESTING ADDS

The VAPT report identified the missing headers. This test **weaponised them** and confirmed exploitability against the live production site — and additionally established that no JavaScript fallback defence exists, which header inspection alone cannot determine.

PROOF OF CONCEPT

A minimal attacker page hosted on an unrelated origin embeds the production site and renders an overlay above it:

```
<iframe id="f" src="https://re-new-mocha.vercel.app/"
      style="width:100%;height:800px;border:0"></iframe>
<div class="overlay">↓ attacker-controlled overlay sits on top of the real site ↓</div>
<script>
  document.getElementById('f').addEventListener('load', () => {
    console.log('[PoC] iframe load event fired – target is framable');
  });
</script>
```

"FRAMED SUCCESSFULLY – no X-Frame-Options / frame-ancestors enforced"

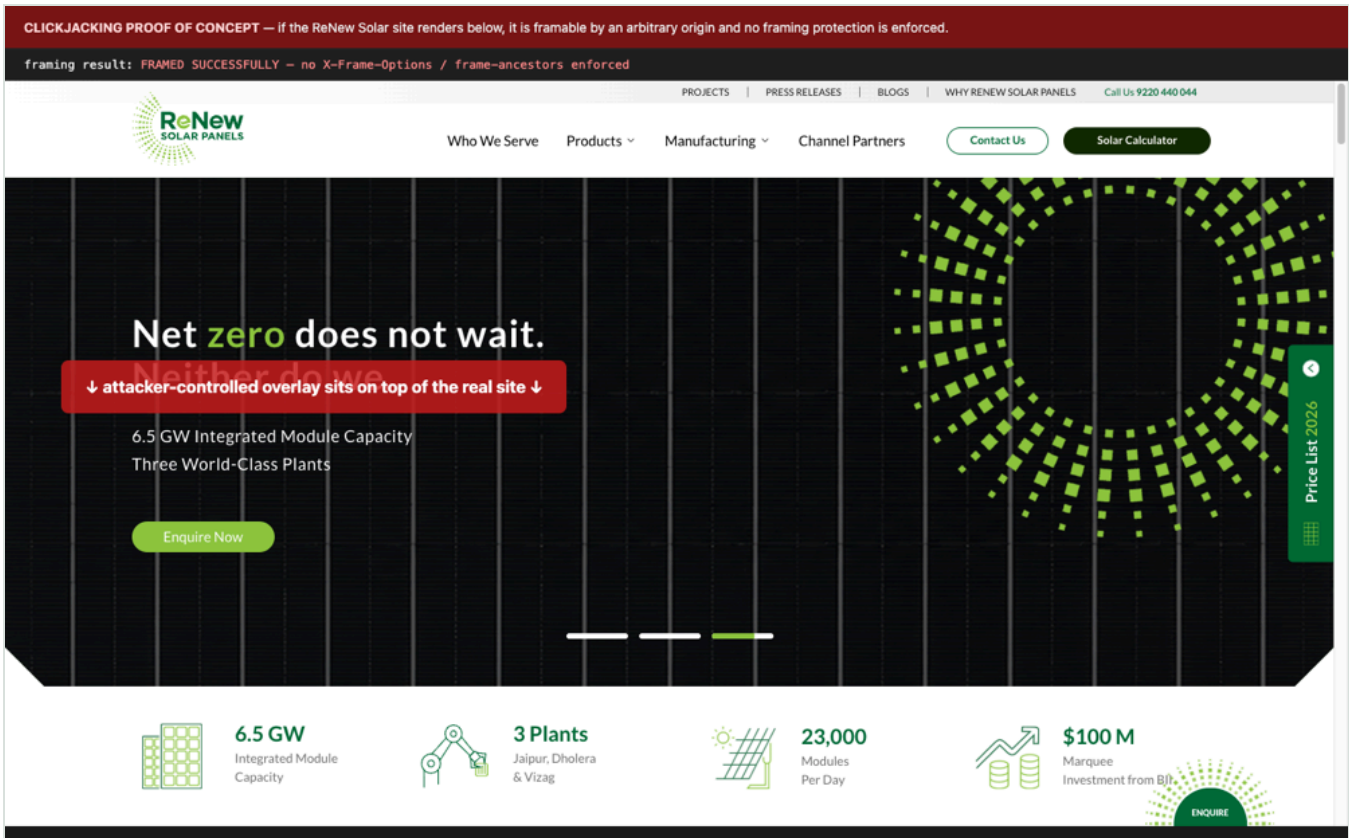


Figure 1 — The live production site rendered inside an attacker-controlled frame from an unrelated origin, with an overlay positioned above the hero content. Captured 17 August 2026 against `https://re-new-mocha.vercel.app/`.

NO FALLBACK DEFENCE EXISTS

Some applications compensate for missing headers with JavaScript frame-busting. This one does not — verified across both the source tree and the compiled production bundles:

```
frame-busting occurrences in source + built bundles: 0
```

IMPACT

An attacker can present the genuine, fully-functional ReNew Solar site inside a page they control and overlay it with their own interface. Realistic abuse for a brand of this kind includes a fraudulent "authorised dealer registration" or "subsidy application" wrapper that borrows the site's visual legitimacy to harvest details, or transparent overlays that capture interaction intended for the real page. Because the site holds no session and no authentication, there is no account to compromise — the damage is **brand impersonation and user deception** rather than data breach, which is why this is rated Medium rather than High.

REMEDIATION

Ship framing protection at the Vercel platform layer. Note that `headers()` in `next.config.ts` has no effect under `output: "export"` — it must be `vercel.json`:

```
{
  "headers": [{
    "source": "/(.*)",
    "headers": [
      { "key": "X-Frame-Options", "value": "DENY" },
      { "key": "Content-Security-Policy", "value": "frame-ancestors 'none'" }
    ]
  }]
}
```

Ship both: **frame-ancestors** is the modern control and takes precedence where supported, while **X-Frame-Options** covers older agents. Retest by re-running the PoC — the iframe should fail to render. The full recommended header set is in VAPT-01.

DAST-03 Required-field validation passes with the field empty

MEDIUM

CWE	OWASP	METHOD	CROSS-REF
CWE-1173 — Improper Use of Validation Framework	A04:2021 — Insecure Design	Live DOM + constraint API	SAST-02

EVIDENCE

The rendered DOM shows the mandatory "Requirement type" control is the only field in the form not carrying the constraint:

```
      required: true
      required: true      # genuinely optional
      required: true
      required: false     # marked mandatory in the UI
```

Confirmed against the browser's own constraint-validation API. With `requirement` left empty, the form reported itself valid and submitted:

```
"formValidatedAsComplete_withEmptyRequirement": true,
```

IMPACT

Users see a red asterisk denoting a mandatory field, yet the form submits without it. The consequence today is contained by DAST-02 — nothing is transmitted. The finding matters because it creates a **false assumption of validation** for whoever builds the endpoint: server-side logic written on the assumption that `requirement` is always populated will be wrong from the first request.

REMEDIATION

Note that `required` on a `type="hidden"` input is ignored by browsers per specification, so renaming the attribute will not fix this. Use a visually-hidden *text* input, which does participate in constraint validation — full code is given in SAST-02. Regardless of the client fix, the endpoint must validate every field server-side.

DAST-04 Calculator returns identical output for every state**MEDIUM**

CWE	OWASP	METHOD	CROSS-REF
CWE-682 — Incorrect Calculation	A04:2021 — Insecure Design	Driven UI interaction	SAST-01

EVIDENCE

The state selector was driven through four states with genuinely different tariffs. The displayed unit cost and every output figure were **byte-identical** in all four cases:

STATE SELECTED	TRUE TARIFF	UNIT COST SHOWN	ANNUAL SAVINGS	LIFETIME
Maharashtra	₹11.00	₹8.00	₹48,000	₹14,40,000
Ladakh	₹5.20	₹8.00	₹48,000	₹14,40,000
Kerala	₹7.10	₹8.00	₹48,000	₹14,40,000
Delhi	₹8.00	₹8.00	₹48,000	₹14,40,000

```
# Identical for all four states – the selector demonstrably does not affect the result
```

```
Estimated savings at ₹8.00/kWh.
```

IMPACT

Consumers are shown state-specific framing over a flat, state-independent calculation. Per the quantification in SAST-01, savings are overstated by up to **53.8%** (Ladakh) and understated by **27.3%** for Maharashtra — the page's own default state. The correct per-state tariff data already exists in the component and is simply never read.

REMEDIATION

Wire `STATE_TARIFFS` into the unit cost on state change; see SAST-01 for the code. Retest by repeating the four-state comparison above and confirming the outputs diverge.

DAST-05 Declared numeric bounds not enforced at runtime

LOW

CWE	OWASP	METHOD	CROSS-REF
CWE-1284 — Improper Validation of Quantity	A03:2021 — Injection (validation)	Boundary value injection	SAST-04

EVIDENCE

The consumption field declares `max="5000"` . Values far outside that were accepted and propagated through to the displayed financial projection:

VALUE ENTERED	DECLARED MAX	ACCEPTED	LIFETIME SAVINGS RENDERED
500	5000	Yes	₹14,40,000 (baseline)
999999999	5000	Yes	₹28,79,99,99,97,120
1e12	5000	Yes	₹2,88,00,00,00,00,00,000
-5000	5000	Clamped	₹0 (safe)
abc	5000	Rejected	₹0 (safe)

IMPACT

Malformed input is handled safely — negatives clamp to zero and non-numeric text yields zero, with no `NaN` propagation or crash. Only the upper bound is unguarded, and the result renders a ₹288 trillion figure with the same confidence as a real one. This is a presentation defect today, since the calculation is client-side on a static site and cannot exhaust any shared resource. It is reported because these values become **API inputs** the moment DAST-02 is remediated, where missing upper bounds become a resource-consumption concern (OWASP API4:2023).

REMEDIATION

Clamp in `lib/solar.ts` rather than the component, so any future server-side caller inherits the guarantee. Code in SAST-04.

DAST-06 Security headers absent on all six routes tested

LOW

CWE	OWASP	METHOD	CROSS-REF
CWE-693 — Protection Mechanism Failure	A05:2021 — Misconfiguration	Per-route header enumeration	VAPT-01

WHAT DYNAMIC TESTING ADDS

The VAPT tested the homepage. This test enumerated **every route including the 404 handler**, confirming the gap is uniform rather than a single-route oversight — which matters, because a partial rollout of the fix would be easy to mistake for a complete one.

```
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy
CSP, XFO, XCT0, Referrer-Policy, Permissions-Policy

strict-transport-security: max-age=63072000; includeSubDomains; preload ← present on all
set-cookie: (none on any route — no cookie attack surface)
```

The missing framing header is the exploited component and is reported separately as DAST-01. The remainder — MIME-sniffing protection, referrer control and permissions policy — carry no demonstrated exploit here but remove defence-in-depth. `X-Content-Type-Options` is the most relevant of the three given the 16 user-downloadable PDFs served from `/downloads`.

REMEDIATION

Apply the full header set via `vercel.json` — see VAPT-01 for the complete policy.

DAST-07 Wildcard CORS on every response including documents

LOW

CVSS V3.1	CWE	METHOD	CROSS-REF
3.1 — Low	CWE-942 — Permissive CORS Policy	Header enumeration	VAPT-05

EVIDENCE & IMPACT

All six routes return `access-control-allow-origin: *`, permitting any origin to read response bodies cross-origin. Dynamic testing confirms the two conditions that would make this dangerous are both **absent**: no route issued a `Set-Cookie`, and runtime inspection found no client-side session state of any kind. Every response is identical public marketing content, so an attacker gains nothing cross-origin that they cannot already fetch directly.

It is reported as a latent misconfiguration. The wildcard is applied blanket at the platform level, so an authenticated area or a form-handling endpoint added later would inherit it and expose genuinely sensitive responses.

REMEDIATION

Scope the header to the assets that need it (fonts, media) and remove it from document responses — see VAPT-05. The endpoint built for DAST-02 must use an explicit origin allow-list and must never inherit this wildcard.

DAST-08 **React Server Component payloads publicly retrievable**

INFO

CWE	SENSITIVE DATA	METHOD	ENV
CWE-200 — Information Exposure	None found	Artifact enumeration	Production

EVIDENCE & IMPACT

The static export publishes React Server Component payload files alongside the HTML. They are retrievable without restriction:



These contain no sensitive data. They hold the serialised component tree for pages whose content is already public, plus internal chunk filenames and component structure — the same information derivable from the shipped JavaScript. They are a normal and expected product of the Next.js App Router static export, not a misconfiguration.

Recorded for two reasons: they are non-obvious and worth knowing exist, and they slightly ease an attacker's job in mapping application structure. If page content that is *not* intended to be public is ever added to this site, it would appear here in plain text as well as in the HTML.

REMEDIATION

No action required. Treat as a reminder that everything reachable in the export is public: any future gated content needs real access control, not merely absence of a link.

DAST-09 Shared platform wildcard certificate on preview domain

INFO

CWE	TLS	METHOD	ENV
N/A — observation	1.3, valid chain	TLS inspection	Production

EVIDENCE

```
Protocol : TLSv1.3
Cipher   : TLS_AES_128_GCM_SHA256
Verify return code: 0 (ok)

                                     ← shared platform wildcard

# Legacy protocol negotiation
REJECTED      REJECTED      NEGOTIATED
```

IMPACT

The TLS configuration is correct and requires no remediation. TLS 1.3 negotiated, obsolete protocols refused, certificate chain valid, HSTS preloaded with a two-year max-age.

The single observation is that the certificate is Vercel's shared *.vercel.app wildcard rather than one issued for a ReNew-controlled domain — expected for a preview deployment. It is noted because it confirms this assessment targeted a preview environment. When the site moves to a ReNew production domain, the certificate, HSTS scope, and every header finding in this report must be re-verified against that environment, which may sit behind different edge configuration.

5 Controls Verified Under Attack

Behaviour observed while the application was actively being attacked. These passed, and are recorded so a future change does not silently regress them.

5.1 Injection resistance — the one real input

The URL fragment is the only attacker-controllable input in the application. Twelve payloads were fired at it in a live browser. **The positive control passed first** — two valid identifiers selected two different tabs, proving the handler was live:

```
"handlerIsLive": true,           # valid ids selected DIFFERENT tabs – the code path ran
"anyXssFired": false,
"anyInjection": false,
"anyProtoPollution": false }
```

PAYLOAD	CLASS	RESULT	TAB AFTER
g12r-topcon	Valid control	ACCEPTED	G12R TOPCon Bifacial
m10r-topcon	Valid control	ACCEPTED	M10R TOPCon
	XSS	REJECTED	unchanged
"><script>...</script>	XSS	REJECTED	unchanged
<svg/onload=...>	XSS	REJECTED	unchanged
javascript:...	XSS	REJECTED	unchanged
'-alert(1)-'	XSS	REJECTED	unchanged
%3Cscript%3E...%3C/script%3E	XSS (encoded)	REJECTED	unchanged
../../../../etc/passwd	Traversal	REJECTED	unchanged
__proto__	Proto pollution	REJECTED	unchanged
constructor	Proto pollution	REJECTED	unchanged
g12r-topcon	Prefix bypass	REJECTED	unchanged

WHY THIS HOLDS — AND THE DETAIL WORTH PRESERVING

The handler validates against a closed `Set` of known identifiers. Two results are worth calling out. The **valid-prefix payload was rejected**, proving exact matching rather than a weaker prefix or substring test. And `__proto__` and `constructor` were rejected because the code uses `Set.has()` rather than a plain-object lookup — had it used `MAP[hash]`, those two payloads would have returned inherited `Object.prototype` members and behaved very differently. That choice is load-bearing and should survive any future refactor.

5.2 Runtime hygiene

CHECK	RESULT	OBSERVED
Third-party network egress	CLEAN	Zero — no analytics, tag manager, or beacon
Client-side storage	CLEAN	<code>localStorage</code> and <code>sessionStorage</code> both empty
Cookies	CLEAN	<code>document.cookie</code> empty; no <code>Set-Cookie</code> on any route
Service workers	CLEAN	None registered — no offline cache to poison
Console output	CLEAN	Zero messages — no errors, warnings, or debug leakage
Mixed content	CLEAN	0 insecure <code>http://</code> subresources
Reverse tabnabbing	CLEAN	0 of 17 external links missing <code>noopener</code>
<code>javascript:</code> URIs	CLEAN	0 across 70 rendered links
Inline event handlers	CLEAN	0 <code>on*</code> attributes in the live DOM
Reflected content in 404	CLEAN	Body byte-identical (43,865) across all probes

5.3 Edge and protocol behaviour

CHECK	RESULT	OBSERVED
Mutating HTTP methods	BLOCKED	POST/PUT/DELETE/PATCH/PROPFIND → 405
TRACE (Cross-Site Tracing)	BLOCKED	405
Encoded path traversal	BLOCKED	400 at the edge, before content
HTTPS enforcement	ENFORCED	Plain HTTP → 308 to HTTPS
TLS version floor	CORRECT	1.3 negotiated; 1.0 and 1.1 refused
HSTS	CORRECT	2-year max-age, <code>includeSubDomains</code> , <code>preload</code>
Bot / abuse mitigation	ACTIVE	Challenge engaged on scripted probing; real browsers unaffected

ON THE MITIGATION LAYER

The platform's challenge response is a genuine control and is recorded as a positive. It engaged on this assessment's scripted traffic, persisted for the source IP beyond seven minutes, and left real browser traffic untouched — which is the correct discrimination. It raises the cost of automated scanning and content scraping against the site. It is *not* a substitute for the response headers in DAST-01 and DAST-06: it did not impede the clickjacking exploit at all, because that attack runs entirely in a victim's browser and never touches the platform's request path.

6 Remediation & Retest

#	ID	ACTION	PRIORITY	EFFORT
1	DAST-01	Ship <code>X-Frame-Options: DENY + frame-ancestors 'none'</code> — closes the confirmed exploit	IMMEDIATE	30 min
2	DAST-02	Remove the unsupported 24-hour promise pending the endpoint	IMMEDIATE	< 1 hour
3	DAST-06	Apply the remaining header set across all routes	IMMEDIATE	2 hours
4	DAST-04	Wire the state tariff table into the calculation	1 WEEK	2–3 hours
5	DAST-02	Build the endpoint; derive success from the server response	2 WEEKS	2–3 days
6	DAST-03	Enforce the required constraint on the dropdown	2 WEEKS	2 hours
7	DAST-05	Clamp numeric bounds in the domain layer	2 WEEKS	1 hour
8	DAST-07	Scope CORS away from the wildcard	1 MONTH	1 hour

SEQUENCING

Items 6 and 7 must land *with or before* item 5. Both are validation controls whose absence is currently harmless only because nothing is transmitted. Shipping the endpoint first means it receives incomplete and unbounded input from its first request.

6.1 Retest procedure

Each finding has a concrete pass condition. Re-run these after remediation:

ID	TEST	PASS CONDITION
DAST-01	Re-run the clickjacking PoC from Section 4	iframe fails to render; verdict reads BLOCKED
DAST-02	Submit a test enquiry with egress instrumentation	<code>egressCount > 0</code> ; enquiry arrives in the destination system; UI shows a real error when the endpoint is stopped
DAST-03	<code>form.checkValidity()</code> with requirement empty	returns <code>false</code> ; submission blocked
DAST-04	Compare output across Maharashtra / Ladakh / Kerala	unit cost and savings differ per state
DAST-05	Enter <code>999999999</code> in the consumption field	value clamped; no absurd figure rendered
DAST-06	Enumerate headers on all six routes	all five headers present on every route including 404
DAST-07	Inspect <code>access-control-allow-origin</code> on /	no wildcard on document responses

```
# Fast header regression check – expect all five present
```

Run header checks from a browser or at low volume — the platform's mitigation layer will challenge rapid scripted probing, as it did during this assessment.

7 Appendix

7.1 Dynamic test coverage

TEST CATEGORY (OWASP WSTG)	RESULT	NOTES
Configuration & deployment	FINDINGS	DAST-06, DAST-07, DAST-08
HTTP methods	PASS	12 verbs tested; all mutating verbs blocked
Transport security	PASS	TLS 1.3; legacy refused; HSTS preloaded
Clickjacking	EXPLOITED	DAST-01 — working PoC against production
Reflected XSS	PASS	No reflection in 404 or any route
DOM-based XSS	PASS	7 payloads, control-verified, all rejected
Client-side injection	PASS	Prototype pollution and prefix bypass rejected
Path traversal	PASS	Blocked at edge (400) and in app
Input validation	FINDINGS	DAST-03, DAST-05
Business logic	FINDINGS	DAST-02, DAST-04
Session management	N/A	No cookies or sessions — verified at runtime
Authentication / authorization	N/A	No authentication mechanism exists
CSRF	N/A	No state-changing request; no cookie auth
Server-side injection (SQLi, cmd)	N/A	No server-side runtime — static export
File upload	N/A	No upload capability
Denial of service	EXCLUDED	Out of scope by rules of engagement

7.2 Tooling

TOOL	USE
Chrome DevTools Protocol	Browser automation, runtime instrumentation, console and network capture
Chromium (headless)	Real rendering engine executing the production React runtime
curl	HTTP method enumeration, header inspection, redirect testing
openssl s_client	TLS version, cipher and certificate chain inspection
next build	Reproducing the deployed artifact locally from <code>a4f4020</code>
Custom PoC harness	Clickjacking exploit; egress interception shims

7.3 Limitations

- **Point-in-time.** Reflects the deployment and build `a4f4020` as observed on 17 August 2026.
- **Preview environment.** The target is a Vercel preview domain on a shared wildcard certificate. A move to a ReNew-controlled production domain requires re-verification of every header and TLS finding against that environment.
- **Mitigation interference.** The platform's challenge layer blocked scripted access to production partway through testing. Remaining production checks were completed through the browser. Client-side testing was performed against a locally-built instance of the same commit; because the deployment is a pure static export, those results transfer, but they were not re-executed against production.
- **No authenticated testing.** There is no authentication to test.
- **DoS and load testing excluded** by the rules of engagement.
- **Third-party destinations** linked from the site were catalogued but not tested.

END OF REPORT

ReNew Solar Marketing Website — DAST Report v1.0 — 17 August 2026. 9 findings: 0 Critical, 1 High, 3 Medium, 3 Low, 2 Informational. 1 confirmed exploit (clickjacking). 25 attack payloads fired; 24 repelled. Retest recommended after roadmap items 1–3.